

Morse-LiDAR

Полный конспект математики, алгоритмов и методологии проекта

Версия документа: 1.0 · 25 сентября 2026 · репозиторий triangulation

Короткая идея. Проект превращает LiDAR-скан в триангулированную поверхность, выбирает на ней скалярное поле, извлекает дискретную топологическую структуру Морса–Смейла и сравнивает сканы по устойчивым persistence-диаграммам. Главная прикладная гипотеза: структурные изменения объекта можно обнаруживать с малым количеством размеченных данных и объяснять через изменение топологических особенностей.

Содержание и статус утверждений

- 1 Постановка задачи и объект исследования
- 2 Геометрическая модель и входные данные
- 3 Скалярные поля и дискретная дифференциальная геометрия
- 4 Дискретная теория Морса
- 5 Сепаратрисы и комплекс Морса–Смейла
- 6 Persistent homology и фильтрация шума
- 7 Итоговый дескриптор и сравнение
- 8 Методология эксперимента
- 9 Воспроизводимость и контракт данных
- 10 Ограничения, риски и план валидации
- 11 Прикладное позиционирование

Строгие математические части: определения комплексов, Euler-характеристика, lower/upper link, lower-star-фильтрация, persistence и bottleneck distance при корректных входных данных.

Численные аппроксимации: растеризация, Delaunay/Poisson-реконструкция, дискретные операторы кривизны, RANSAC, PCA и эвристическая трассировка сепаратрис.

Инженерные предположения: поверхность является графом высоты или достаточно хорошо аппроксимируется им; единицы измерения внутри пары сканов совпадают; параметры обработки одинаковы.

1. Постановка задачи

1.1. Вход и выход

Входом служит либо конечное облако точек $P = \{p_i \in \mathbb{R}^3\}$, либо двумерная карта глубины $z = f(x, y)$. Выходом является JSON-дескриптор:

- критические точки: положение, тип, значение скаляра, persistence;
- сепаратрисы: направленные полилинии между критическими точками;
- счётчики min/saddle/max и Euler-проверка;
- persistence diagram по размерностям гомологии;
- метаданные обработки, геометрии, единиц и версии схемы.

Для двух совместимых отчётов вычисляется расстояние между persistence-диаграммами. Для серии сканов экспериментальный модуль дополнительно разделяет пары «тот же объект» и «разные объекты».

1.2. Научная гипотеза

Низкоперсистентные особенности в основном вызваны шумом измерения и дискретизацией, а высокоперсистентные особенности отражают устойчивую геометрию или структурное изменение. Тогда после выбора скалярного поля f и порога τ расстояние между повторными сканами должно быть меньше расстояния между сканом до и после изменения.

H1: $d(\text{repeat}_i, \text{repeat}_j) < d(\text{before}, \text{after})$

H2: $d(\text{same object}) < d(\text{different object})$

H3: устойчивость зависит от f , τ , плотности, шума и реконструкции

Проект не утверждает, что топологический дескриптор универсально превосходит нейросетевые признаки. Его ниша: интерпретируемость, малые выборки, обнаружение изменений и контроль инвариантов.

2. Геометрическая модель

2.1. Треугольный комплекс

Поверхность представляется ориентированным треугольным комплексом $K = (V, E, F)$, где V — вершины, E — рёбра, F — грани. Каждая вершина имеет координату $x_v = (x_v, y_v, z_v)$ и одно скалярное значение $f(v)$.

$$\chi(K) = |V| - |E| + |F|$$

closed $\Leftrightarrow$ каждое ребро инцидентно ровно двум граням

boundary edge $\Leftrightarrow$ ребро инцидентно одной грани

Для замкнутой ориентируемой поверхности Euler-характеристика связана с родом: $\chi = 2 - 2g$. Для тора $\chi = 0$. Для открытой поверхности обычное равенство Морса меняется граничными членами, поэтому проект не применяет закрытую формулу к открытому растру.

2.2. Регулярный растр

Для depth map размера $R \times C$ вершина с индексом (r, c) получает координаты:

$$x = x_0 + c \cdot \Delta x, \quad y = y_0 + r \cdot \Delta y, \quad z = \text{depth}[r, c]$$

Каждая прямоугольная ячейка раскладывается на два треугольника. В periodic-режиме индексы по краям берутся по модулю, что создаёт закрытую сетку для тестовых тороидальных данных. Для реальных сканов periodic-режим запрещён или выдаёт предупреждение: склейка краёв создаёт искусственный шов.

2.3. Облако точек и проектирование

Для iPhone/ARKit ось Y обычно является вертикальной. Преобразование --up-axis y переставляет оси так, чтобы алгоритм работал с Z как высотой. Растризация:

- 1 выбираются две горизонтальные координаты и одна depth-координата;
- 2 bounding box делится на регулярную сетку;
- 3 в каждой ячейке берётся медиана глубины;
- 4 пустые ячейки заполняются ближайшим известным значением;
- 5 сохраняются coverage, bounds и метрический spacing.

Медиана снижает влияние выбросов внутри пикселя. Заполнение ближайшим значением — инженерная мера для

получения полного растра, но оно может создавать плато и искусственные области; coverage обязательно нужно анализировать.

2.4. Delaunay и Poisson

Delaunay: триангуляция выполняется в выбранной 2D-проекции, а третья координата сохраняется как скаляр. Это открытая 2.5D-поверхность. Ориентация граней исправляется так, чтобы нормаль имела положительную компоненту по scalar-axis. Poisson: Open3D восстанавливает полноценную 3D-поверхность по нормальям облака. Обрезка вершин низкой плотности уменьшает галлюцинированные области, но открывает сетку. Для ТТК, которому нужна закрытая поверхность, используется density_quantile = 0, что в свою очередь может сохранить нежелательные области.

3. Скалярные поля и кривизна

3.1. Высота

Поле $f(v)=z_v$ наиболее простое и чувствительно к позе сканера. Оно подходит для повторных сканов при одинаковой нормализации, но не является инвариантным к произвольному вращению объекта.

3.2. Бариецентрическая площадь

Для треугольника (a,b,c) площадь равна $A_f = 1/2 \|(b-a) \times (c-a)\|$. Вершине присваивается треть площади каждой инцидентной грани:

$$A_v = (1/3) \cdot \sum_{f \ni v} A_f$$

Деление на A_v принципиально: без него дискретная гауссова кривизна была бы интегральной и менялась бы при изменении плотности сетки.

3.3. Гауссова кривизна

Для внутренней вершины дискретная формула angle defect:

$$K_v = (2\pi - \sum_{f \ni v} \theta_{f,v}) / A_v$$

Для граничной вершины целевой угол заменяется на π . Однако сырая оценка на ободке нестабильна. В проекте значения на границе не используются как исходные: внутренние оценки распространяются по слоям графового расстояния, каждый новый слой получает среднее уже известных соседей.

3.4. Средняя кривизна

Пусть $\cot \alpha_{ij}$ и $\cot \beta_{ij}$ — котангенсы углов напротив ребра (i,j). Котангенсный дискретный оператор Лапласа–Бельтрами:

$$L(v_i) = [1/(2A_i)] \cdot \sum_{j \in N(i)} (\cot \alpha_{ij} + \cot \beta_{ij}) (x_j - x_i)$$

$$L(x_i) \approx -2 H_i n_i$$

$$H_i = -(L(x_i) \cdot n_i)/2$$

Реализация эквивалентно использует накопленную cotangent-сумму и деление на $4A_i$. Знак зависит от ориентации нормали. Для сеток проекта нормаль выбирается в сторону +Z; поэтому купол и чаша имеют противоположные знаки.

3.5. Shape index

Главные кривизны восстанавливаются как $k_{1,2} = H \pm \sqrt{H^2 - K}$. Затем используется индекс Кендеринка:

$$S = (2/\pi) \cdot \text{atan2}(k_1+k_2, k_1-k_2)$$

$$S = (2/\pi) \cdot \text{atan2}(2H, 2\sqrt{H^2-K}) \in [-1, 1]$$

Геометрия

H

K

Shape index

Купол

положительная

положительная

+1

Седло

около 0

отрицательная

около 0

Чаша

отрицательная

положительная

-1

Плоскость

0

0

0

Кривизна метрична: K имеет единицы 1/длина², H — 1/длина. Поэтому spacing и единицы входного облака должны быть корректны.

4. Дискретная теория Морса

4.1. Simulation of Simplicity

Плато и совпадающие значения нарушают строгий порядок. Проект задаёт детерминированный порядок:

$$u < v \Leftrightarrow f(u) < f(v) \text{ или } [f(u)=f(v) \text{ и } \text{index}(u) < \text{index}(v)]$$

Этот порядок используется одновременно классификатором, persistence и трассировкой. Значит, алгоритмы не расходятся на плато только из-за разных способов сравнения.

4.2. Link вершины

Link вершины v состоит из соседних вершин и рёбер, возникающих в гранях, содержащих v. Он разделяется по порядку SoS:

$$L^-(v) = \{u \in N(v) : u < v\}$$

$$L^+(v) = \{u \in N(v) : v < u\}$$

В проекте считаются связные компоненты индуцированных подграфов нижнего и верхнего link. Обозначим их числа через c^- и c^+ .

4.3. Классификация

Условие

Тип

Интуиция

$c^- = 0$

минимум

нет ниже лежащих направлений

$c^+ = 0$

максимум

нет выше лежащих направлений

$c^- \geq 2$ или $c^+ \geq 2$

седло

link распадается на несколько ветвей

остальное

регулярная

локально один поток входа и выхода

Для седла мультипликативность задаётся $\max(c^-, c^+) - 1$. Простое седло обычно имеет две нисходящие и две восходящие ветви; monkey saddle может иметь большую кратность.

4.4. Euler-инвариант

Для замкнутой поверхности проверяется дискретное Morse-соотношение:

$$\chi(K) = \# \min - \sum \text{saddle multiplicity}(s) + \# \max$$

Проверка важна как fail-fast тест: нарушение обычно означает неверную ориентацию, топологически испорченную сетку, неправильную классификацию или несогласованную обработку сёдел. Для открытой поверхности проект не выдаёт ложное утверждение: euler_characteristic может быть null.

5. Сепаратрисы и Morse–Smale complex

5.1. Базовая трассировка

Эвристический backend идёт по соседям строго монотонно в SoS-порядке. Из каждой компоненты нижнего link седла трассируется нисходящая u-дуга к минимуму; из каждой компоненты верхнего link — восходящая s-дуга к максимуму.

u-arc: order уменьшается до локального минимума

s-arc: order увеличивается до локального максимума

Строгая монотонность гарантирует отсутствие циклов в walk и конечность трассировки. На открытой поверхности дуга может закончиться на замаскированной границе; это явно маркируется ends_on_boundary, а не удаляется молча.

Статус: это детерминированный baseline, а не полный дискретный градиентный Morse–Smale complex. Он пригоден для тестов, JSON-контракта и простых сравнений, но не заменяет production-реализацию TTK.

5.2. TTK backend

TTK/ParaView вычисляет критические точки, полилинии сепаратрис и Morse–Smale quadrangulation. Проект запускает rpython как отдельный процесс, проверяет массивы версии TTK и преобразует результат в общий контракт.

Для закрытой поверхности каждая Morse–Smale ячейка имеет схему min–saddle–max–saddle. Она разрезается новой t-дугой на два треугольника дуального графа. Узел графа должен иметь ровно три инцидентных ребра: s, u, t.

$\deg(v) = 3$

$\text{colors}(v) = \{s, u, t\}$

каждое dual edge $\leftrightarrow$ ровно одна общая primal arc

Проверки графа связывают абстрактную комбинаторику с реальными дугами исходной триангуляции и не позволяют случайно заменить топологический объект только списком пар критических точек.

6. Persistent homology

6.1. Фильтрация lower-star

Для скалярного поля на вершинах simplex получает значение максимума своих вершин:

$$F(\sigma) = \max \{ f(v) : v \in \sigma \}$$

Это lower-star filtration: при росте уровня a в комплекс добавляются все симплексы, у которых $F(\sigma) \leq a$. В коде вершины, затем рёбра, затем треугольники вставляются явно. Иначе GUDHI мог бы создать ребро с фильтрацией треугольника, что нарушило бы lower-star-структуру.

Для maxima поле инвертируется: $f \mapsto -f$. Для mesh persistence используется именно та же триангуляция, что и для классификатора, а ties разбиваются SoS-рангом.

6.2. Интервалы

Persistence interval имеет вид (b, d) , где класс рождается на уровне b и исчезает на уровне d . Его persistence:

$$\text{pers} = d - b; \quad \text{pers} = \infty \text{ для essential class}$$

На поверхности размерность 0 соответствует компонентам связности и парам minimum–saddle; размерность 1 — циклам и парам saddle–maximum; старшая размерность содержит глобальный максимум на закрытой поверхности. Род поверхности отражается числом essential 1-dimensional classes.

6.3. Связывание с критическими вершинами

GUDHI-пара переводится в вершины, создающие birth/death simplex. Для каждой критической вершины хранится максимальная persistence среди пар, в которых она участвует. Сёдла учитываются с кратностью: одно k -кратное седло может принадлежать k парам.

Порог τ оставляет пары с $\text{pers} \geq \tau$ и формирует persistent_counts. Это полезная статистика, но не полноценная cancellation.

Важно. В текущей реализации фильтрация удаляет низкоперсистентные пары только из счётчиков и аннотаций.

Сепаратрисы не перестраиваются. Для настоящего упрощённого MSC нужно отменять пары и перенаправлять дуги.

6.4. Bottleneck distance

Для persistence diagrams D_1, D_2 bottleneck distance минимизирует максимальное L_∞ -расстояние при сопоставлении точек, разрешая сопоставление с диагональю. Реализация вызывает GUDHI для конечных точек и отдельно проверяет essential points:

$$d_B(D_1, D_2) = \inf_\gamma \sup_{p \in D_1} \|p - \gamma(p)\|_\infty$$

Теоретически для подходящей фильтрации выполняется stability bound:

$$d_B(D(f), D(g)) \leq \|f - g\|_\infty$$

Это не означает стабильность самого комбинаторного графа: маленький шум может поменять число критических точек и связи. Поэтому persistence diagram является более надёжной численной частью дескриптора, чем сырые counts.

7. Дескриптор и контракт JSON

7.1. Состав

Компонент

Смысл

Надёжность

critical_points

типы и геометрия особых вершин

зависит от шума и threshold

separatrices

структурные пути между особенностями

ТТК выше baseline

persistent_counts

сколько критических элементов пережило t

полезно для мониторинга

persistence_diagram

устойчивый численный инвариант фильтрации

основная метрика сравнения

colored_graph

комбинаторный dual MSC-граф

только закрытая ТТК-сцена

Идентификаторы локальны одному отчёту и не являются correspondences между сканами. Сравнение разрешено только при совпадении schema version, scalar, geometry, surface, параметров обработки и persistence threshold.

7.2. Что сравнивается

На текущем этапе основное расстояние — bottleneck по диаграммам. Counts выводятся для интерпретации, но не являются стабильным расстоянием. Расстояние между colored graphs, graph edit distance и WL-kernel пока не реализованы.

Практическое решение для пары отчётов:

- 1 проверить совместимость метаданных;
- 2 сравнить диаграммы по каждой размерности;
- 3 взять максимум расстояний как bottleneck_max;
- 4 посмотреть equality и различия persistent_counts;
- 5 для объяснения показать изменившиеся критические точки и локальные области скана.

8. Полный вычислительный pipeline

- 1 Загрузка: NPY, XYZ, CSV, ASCII/binary PLY, PCD, LAS/LAZ через optional dependencies.
- 2 Нормализация: перестановка up-axis, опционально PCA alignment, voxel downsampling.
- 3 Удаление фона: RANSAC удаляет доминирующую плоскость пола, стола или стены.
- 4 Реконструкция: raster, Delaunay или Poisson.
- 5 Предобработка: median/Gaussian smoothing только для raster; режим wrap для periodic и nearest для open.
- 6 Скаляризация: height, Gaussian curvature, mean curvature или shape index.
- 7 Топология: соседства, link-компоненты, SoS, critical points.
- 8 Сепаратрисы: heuristic monotone baseline или ТТК.
- 9 Persistence: lower-star на том же mesh, pairing и аннотация.
- 10 Валидация: Euler, JSON schema, границы и согласованность графа.
- 11 Сравнение: bottleneck, counts и persistent counts.

9. Методология эксперимента

9.1. Единица эксперимента

Эксперимент задаётся JSON-manifest: единицы измерения, фиксированные параметры и список файлов с object labels.

Требуются минимум два скана одного объекта. Результаты содержат SHA-256 входов, effective parameters, версии Python/dependencies и hashes исходных модулей.

Для n сканов строятся все $n(n-1)/2$ пары. Метка same_object выводится только из manifest и не угадывается по имени файла.

9.2. Дизайн датасета

Минимальный пилот: 3–4 объекта размером примерно 0.3–1.5 м, по 3–5 повторных сканов каждого объекта, разные расстояния, стороны и скорость обхода. Для главного кейса нужны пары «до/после»: добавленная деталь, снятая деталь, вмятина, трещина или смещение элемента.

Для научной оценки полезны ModelNet40/ShapeNet как синтетическая база, ScanObjectNN как реальные сканы, SHREC и TOSCA для форм и деформаций. Реальные iPhone-сканы должны быть отдельным benchmark из-за специфики ARKit fusion.

9.3. Факторы

Фактор

План уровней

шум

σ как доля диагонали bounding box

разреживание

100%, 50%, 20%, 10%, 5%

поза

случайные повороты $SO(3)$, затем normalization

геометрия

raster, Delaunay, Poisson

скаляр
height, K, H, shape index, в будущем HKS/LB eigenfunctions

т
0, несколько абсолютных уровней, доля диапазона
размер раstra
например 64^2 , 96^2 , 128^2 , 256^2

9.4. Метрики

- Повторяемость: распределение bottleneck для повторов одного объекта.
- Чувствительность к изменению: расстояние before/after и доля обнаруженных изменений.
- Разделимость: ROC-AUC, mAP или отношение межклассового к внутриклассовому расстоянию.
- Классификация: nearest-neighbour accuracy при leave-one-scan-out.
- Топологическая стабильность: доля совпадения persistent counts, число устойчивых критических точек.
- Геометрическая корректность: ошибка кривизны на аналитических поверхностях.
- Инженерные метрики: runtime, память, coverage и процент неуспешных реконструкций.

9.5. Baselines

Чтобы доказать ценность метода, сравнение должно включать хотя бы один геометрический и один learned baseline: FPFH/SHOT или D2, persistence images/landscapes, ShapeDNA, PointNet/DGCNN либо готовый embedding point cloud. Для change detection нужен также простой baseline: разность выровненных depth maps или Chamfer distance.

9.6. Статистика

Не следует делать вывод по одной паре синтетических сканов. Для каждого объекта считать распределения, медианы и доверительные интервалы по повторам. Параметры выбирать на validation subset; тестовые объекты и test changes не использовать для подгонки т. Отдельно показывать чувствительность к единицам, pose, coverage и preprocessing.

10. Воспроизводимость

10.1. Контроль единиц

Все координаты и distance thresholds должны быть в одинаковых единицах. Кривизны автоматически не конвертируются. Если один скан в миллиметрах, а другой в метрах, диаграммы и т физически несопоставимы.

10.2. Контроль параметров

Нельзя сравнивать отчёты, полученные разными scalar, geometry, surface, sigma, voxel size или threshold. JSON contract отклоняет несовместимые параметры. Для публикации нужно архивировать manifest, input hashes, source revision, package versions и результаты.

10.3. Тесты

- плоскость: $K=H=S=0$;
- сфера, цилиндр, купол, чаша и saddle: аналитические знаки и масштабы;
- top: Euler и persistent counts;
- плато: совпадение SoS в classifier/persistence/tracer;
- граница: явные boundary endpoints и curvature extension;
- CLI: invalid combinations и JSON references;
- эксперимент: distinct paths, labels, hashes, all pair counts;
- TTK: required array names, quadrangulation и trivalent colored graph.

В текущей рабочей среде команды python и pytest не были доступны, поэтому локальный запуск тестов не является частью подтверждённых результатов этого документа. CI использует Python 3.10, 3.12 и 3.13 и устанавливает .[dev,topology,signal].

11. Ограничения и риски

Риск

Почему важен

Следующий шаг

2.5D

потеря задней стороны и зависимость от проекции

Poisson/полная 3D модель и pose study

шум кривизны

производные усиливают шум, критических точек тысячи

HKS, Laplace–Beltrami, multi-scale smoothing

открытая граница

закрытая Euler/Morse теория неприменима напрямую

relative Morse theory или явная boundary policy

threshold только в counts

граф не упрощается вместе с persistence

настоящее cancellation MSC

TTK deployment

ParaView/pypython сложнее обычного pip

контейнер/conda runtime и smoke test

нет real benchmark

синтетика не доказывает практическую полезность

iPhone/ScanObjectNN pilot

нет graph distance

комбинаторная часть пока плохо сравнима

graph edit distance/WL kernel

12. Прикладное позиционирование

Наиболее честный продуктовый тезис: интерпретируемый контроль изменений по повторным LiDAR-сканам.

Пользователь получает не только «похож/не похож», но и устойчивые структурные причины: новый максимум, исчезнувшая седловая связь, изменение persistence и локальная геометрия.

Приоритетные вертикали:

- строительный контроль «до/после»;

- инспекция деталей и поверхностей;
- контроль комплектации и отсутствующих элементов;
- мониторинг деформаций и повреждений;
- научная обработка малых коллекций 3D-сканов.

Для акселератора нужно показывать не «теорию Морса», а короткий путь доказательства: два реальных скана, одно намеренное изменение, визуализация области изменения, численное расстояние, повторяемость на нескольких объектах и baseline.

13. Команды и соответствие коду

```
pip install -e '[dev,topology,signal]'
pytest
```

```
morse-lidar --input scan.ply --output scan.json \
  --up-axis y --remove-plane 0.01 --voxel-size 0.01 \
  --geometry delaunay --persistence-threshold 0.01
```

```
morse-lidar-compare first.json second.json
```

```
python -m morse_lidar.experiment examples/repeat_scans.json results/run_01
```

Модуль

Ответственность

pointcloud.py

загрузка, оси, PCA, downsampling, rasterization

surface.py

Delaunay и Poisson

mesh.py

TriMesh, topology cache, areas, normals, SoS

intrinsic.py

K, H, shape index

critical.py

link components и Morse classification

separatrices.py

heuristic monotone arcs

persistence.py

GUDHI, lower-star, pairs, counts

descriptor.py

descriptor и invariants

ttk_backend.py

production MSC adapter

compare.py

bottleneck comparison

experiment.py

reproducible repeat-scan experiment

Итог

Проект представляет собой topology-first pipeline для LiDAR, а не готовую систему распознавания. Математическое ядро состоит из дискретной геометрии поверхности, link-based Morse classification и persistent homology. Методологическое ядро — честное разделение устойчивых и неустойчивых частей, фиксирование единиц и параметров, paired repeat-scan protocol и сравнение с baseline.

Главный критерий успеха: на реальных данных внутриобъектное расстояние должно быть стабильно ниже расстояния для контролируемого изменения, при этом результат должен быть воспроизводимым, объяснимым и не зависеть от случайной плотности сетки.

Документ составлен по реализации и контрактам текущего репозитория. Формулы описывают используемые дискретные аппроксимации; они не заменяют доказательства сходимости на конкретном классе шумных LiDAR-поверхностей.